Interprocess Communication - Part 1

Operating Systems

Universität Innsbruck

Summer Semester 2026

1 Interprocess Communication with Pipes

In this exercise, we will use pipes for communication between processes. We start with unnamed pipes between parent and child processes, and then look at named pipes (FIFOs) that can be opened by independent processes.

You can inspect the relevant manual pages with:

```
man 2 pipe
man 7 pipe
man 7 fifo
```

1.1 Unnamed Pipes

1.1.1 Simple unnamed pipe

Have a look at the following code where we create a pipe:

```c
#include <stdlib.h>
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <string.h>

int main() {
    int     fd[2];
    pid_t   childpid;

    pipe(fd);
    /* Let in = fd[0] and out = fd[1]. Then at the point in code, the
       data streams through the pipe as follows:

        +----------+     in     +--------+
        |  Process | <--------- | Kernel |
        |          | ---------> |        |
        +----------+    out     +--------+

    */

    if((childpid = fork()) == -1) {
        perror("fork");
        exit(1);
    }
    /* Now since we have both a parent and child process, that use
       the same file descriptors, the data flow looks like:

        +----------+     in     +--------+     in     +---------+
        |  Parent  | <--------- | Kernel | ---------> |  Child  |
        |  Process | ---------> |        | <--------- | Process |
        +----------+    out     +--------+    out     +---------+

       In the following code, both the parent and child will close
```

```
        one of the pipe's file descriptors after which the data flow
        looks like:

         +----------+      in      +--------+             +---------+
         |  Parent  | <--------- | Kernel |             |  Child  |
         |  Process |            |        | <--------- | Process |
         +----------+            +--------+     out     +---------+
    */

    if(childpid == 0) {
        /* Child process closes up input side of pipe */
        close(fd[0]);

        char *s = "hello from child\n";
        write (fd[1], s, strlen(s));
    }
    else {
        /* Parent process closes up output side of pipe */
        close(fd[1]);

        char c;
        while (read(fd[0], &c, 1) > 0) {
            putchar(c);
        }
    }
}
```

The call `pipe(fd)` creates two file descriptors: `fd[0]` for reading and `fd[1]` for writing. After `fork()`, both processes inherit both descriptors. Therefore each process must close the end it does not use.

Run:

```
./01_pipe
```

and you should see:

```
hello from child
```

This is the string that the parent process received through the pipe and wrote to the stdout.

1.1.2 Why closing the write end matters

Now let's try a modified version of the code, where the parent does not close the output side of the pipe:

```
#include <stdlib.h>
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <string.h>

int main() {
```

```c
    int     fd[2];
    pid_t   childpid;

    pipe(fd);

    if((childpid = fork()) == -1) {
        perror("fork");
        exit(1);
    }

    if(childpid == 0) {
        /* Child process closes up input side of pipe */
        close(fd[0]);

        char *s = "hello from child\n";
        write (fd[1], s, strlen(s));
    }
    else {
        /* Parent process closes up output side of pipe */
        //close(fd[1]);

        char c;
        while (read(fd[0], &c, 1) > 0) {
            putchar(c);
        }
    }
}
```

Run:

```
./02_pipe
```

You will see the text from the child, but the parent will not terminate. The reason is that `read()` returns `0` only when EOF is reached (errors are indicated by `-1`). Here the parent still keeps a write descriptor open and, as long as that is the case, the stream has not reached EOF and `read()` blocks until it reads the next byte from the pipe.

1.1.3 Using a pipe as stdin for another program

In the next program, the parent writes text to the pipe, while the child executes `sort`. The child's stdin is connected to the pipe using `dup()`. `dup(fd)` creates a duplicate file descriptor that refers to the same underlying open file description as `fd`. The newly created file descriptor is returned and is always the smallest currently unused descriptor number. In this example, we first close descriptor `0` (stdin) in the child, so the next `dup(fd[0])` returns `0`. Therefore the pipe read end becomes the child's stdin.

```c
#include <stdlib.h>
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
```

```c
#include <string.h>

int main()
{
    int     fd[2];
    pid_t   childpid;

    pipe(fd);

    if((childpid = fork()) == -1)
    {
        perror("fork");
        exit(1);
    }

    if(childpid == 0)
    {
        close(fd[1]);

        /* Close up standard input of the child */
        close(0);

        /* Duplicate the input side of pipe to stdin */
        dup(fd[0]);
        execlp("sort", "sort", NULL);
    }
    else
    {
        /* Parent process closes up input side of pipe */
        close(fd[0]);

        char *s = "Bob\nAlice\nCharlie\n";
        write (fd[1], s, strlen(s));

        close(fd[1]);
        waitpid(childpid, NULL, 0);
    }
}
```

Run:

```
./03_pipe
```

output:

```
Alice
Bob
Charlie
```

The important concept is that `execlp()` does not create a new process. It replaces the current process image. The process itself remains the same process (same PID), and its open file descriptors are preserved across `exec` unless they were marked with close-on-exec. Because stdin was redirected before `execlp()`, `sort` reads from the pipe.

1.1.4 Using dup2 for explicit redirection

The following version does the same as before, but uses dup2() instead of dup();. The call dup2(oldfd, newfd) duplicates oldfd exactly onto newfd. If newfd is already open, it is closed first by the kernel. So dup2(fd[0], STDIN_FILENO) directly expresses the intent "make fd[0] the stdin". This is usually clearer and less error-prone than manually doing close(0) followed by dup(fd[0]).

```c
#include <stdlib.h>
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <string.h>

int main()
{
    int     fd[2];
    pid_t   childpid;

    pipe(fd);

    if((childpid = fork()) == -1)
    {
        perror("fork");
        exit(1);
    }

    if(childpid == 0)
    {
        close(fd[1]);

        /* Close stdin, duplicate the input side of pipe to stdin */
        dup2(fd[0], STDIN_FILENO); // Note: STDIN_FILENO == 0
        execlp("sort", "sort", NULL);
    }
    else
    {
        /* Parent process closes up input side of pipe */
        close(fd[0]);

        char *s = "Bob\nAlice\nCharlie\n";
        write (fd[1], s, strlen(s));

        close(fd[1]);
        waitpid(childpid, NULL, 0);
    }
}
```

Run:

```
./04_pipe
```

which gives again:

```
Alice
Bob
Charlie
```

1.2 Named Pipes (FIFOs)

Unnamed pipes are usually used between related processes (for example parent and child). A FIFO, in contrast, has a pathname in the file system, so unrelated processes can communicate by opening the same path.

The following program creates a FIFO called `./my_fifo`, then repeatedly reads a line from it, prints it and reopens the FIFO if it reaches EOF. The call `mkfifo(FIFO_FILE, 0666)` creates a FIFO at that path. The mode `0666` requests read/write permissions for user, group and others, additionally filtered by the current umask. If the FIFO already exists, `mkfifo()` returns `-1` and sets `errno` to `EEXIST`; this example treats that case as non-fatal.

```c
#include <errno.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/stat.h>
#include <unistd.h>

#define FIFO_FILE "./my_fifo"

int main(void)
{
    FILE *fp;
    char readbuf[80];

    /* Create the FIFO if it does not exist */
    umask(0);
    if (mkfifo(FIFO_FILE, 0666) == -1 && errno != EEXIST) {
        perror("mkfifo");
        return EXIT_FAILURE;
    }

    while(1)
    {
        fp = fopen(FIFO_FILE, "r");
        while (fgets(readbuf, 80, fp) != NULL)
        {
            printf("Received string: %s\n", readbuf);
        }
        fclose(fp);
        printf ("End of stream reached\n");
    }

    return(0);
}
```

Run this reader in one terminal:

```
./05_fifo
```

Then use a second terminal as writer:

```
printf "hello\nworld" > my_fifo; sleep 1; printf "hello again" > my_fifo
```

Output:

```
Received string: hello

Received string: world

End of stream reached
Received string: hello again
End of stream reached
```

Here we see `"End of stream reached"` in between because the first writer closes the FIFO, then there is a `sleep(1)` before the second writer opens it again. During that gap, the reader consumes all remaining bytes from the first write and then `fgets()` returns `NULL` due to EOF on that open stream. The reader closes and reopens the FIFO in the next loop iteration, so the message appears between the two write phases.

Now run without sleep:

```
printf "hello\nworld\n" > my_fifo; printf "hello again" > my_fifo
```

Output:

```
Received string: hello

Received string: world

Received string: hello again
End of stream reached
```

Without the sleep, the second writer opens the FIFO quickly after the first one. In this run, the reader still had the FIFO open and continued reading while a writer was available again, so it did not observe an EOF boundary between the two writes. As a result, both write operations were read as one continuous stream and `"End of stream reached"` appeared only once at the end, after the final writer closed.